

PROJET : GENÈVE LATINE

Tableau Synthétique des Spécifications Techniques et Fonctionnelles (V1 & V2)

Composant / Skill	Technologie	Rôle / Fonctionnalité (V1)	Évolution Future (V2)	Règles Clean Code
Frontend Public	Angular (TS) Tailwind CSS v4	Portail de consultation pour les restaurants, événements, annonces d'emploi, services et podcasts en français.	Affichage des profils utilisateurs style Instagram, filtres avancés par quartier de Genève.	Max 150 lignes par fichier composant TS. Pas de requêtes HTTP directes dans les composants (Services isolés).
Frontend Admin	Angular (TS) Tailwind CSS v4	Panel d'administration privé pour créer, modifier, supprimer le contenu et visualiser les statistiques basiques.	Espace membre pour la gestion autonome des abonnements, des annonces payantes et des profils de commerce.	Composants formulaires atomiques et réutilisables. Fichiers de logique de moins de 150 lignes.
Backend API	Node.js Express + TypeScript	API REST robuste. Structure MVC stricte. Vérification des tokens avec le SDK Admin Firebase.	Gestion des abonnements et paiements récurrents intégrés avec l'API Stripe.	Contrôleurs minces (max 50 lignes par méthode). Routes dédiées sans logique métier.
Base de Données	MongoDB Mongoose (ODM)	5 collections spécialisées et indépendantes : restaurants, events, jobs, services, media.	Ajout de collections pour les profils utilisateurs (users) et les abonnements/transactions.	Indexation sur les quartiers (neighborhood) et index de recherche textuelle globale.
Stockage Médias	Firebase Storage	Stockage des logos, photos de couverture des restaurants et bannières d'événements téléversées par l'admin.	Téléversement autonome d'images et photos de produits directement par les commerçants de la communauté.	URLs des médias stockées dans MongoDB. Optimisation du poids des images côté client.
Sécurité & Auth	Firebase Auth	Authentification sécurisée pour le compte administrateur unique afin d'accéder au dashboard.	Système d'inscription et de la connexion pour tous les commerçants et utilisateurs. Rôles admin/user.	Validation de jetons JWT (Firebase ID Token) via middleware dans chaque route d'écriture du backend.
Qualité & VAR	Git + Husky (Hooks)	Interception de 'git commit' pour forcer automatiquement le linter, le compilateur TS et les tests unitaires.	Tests d'intégration automatiques dans des pipelines CI/CD lors de la publication vers production.	Les commits échouent automatiquement en cas de test cassé ou de faute de syntaxe TypeScript.